

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет «Программной инженерии и компьютерной техники»

Сервис-ориентированная архитектура

Лабораторная работа №1

Выполнил:

Григорьев Давид Владимирович и Жук Иван Александрович
Группа: Р3415

Проверил:

Цопа Евгений Алексеевич

Санкт-Петербург, 2026

Содержание

1	Цель работы	3
2	Задание	3
3	Ход работы	3
3.1	Модель данных	3
3.2	Сервис управления организациями	3
3.3	Сервис каталога организаций	4
3.4	Интерактивная документация	5
4	Результат	5
5	Выводы	5
	Приложение А	
		7
	Приложение Б	
		8

1 Цель работы

Разработать спецификации веб-сервисов в формате OpenAPI для управления коллекцией организаций и выполнения дополнительных операций над ней. Описать REST-интерфейсы, ограничения данных и XML-представление, а также подготовить интерактивную документацию Swagger UI.

2 Задание

В соответствии с вариантом 67303 требуется спроектировать два веб-сервиса. Первый управляет коллекцией объектов `organization`: предоставляет базовые операции создания, чтения, изменения и удаления, выборку с фильтрацией, сортировкой и страничным выводом, а также операции статистики. Второй сервис размещается по адресу `/orgdirectory` и предоставляет фильтрацию организаций по числу сотрудников и сортировку по заданному полю. Для обоих сервисов необходимо описать REST API и подготовить Swagger UI.

3 Ход работы

3.1 Модель данных

Основной ресурс — организация. Обязательные поля `name`, `coordinates`, `creationDate`, `type` и `postalAddress` описаны как обязательные. Идентификатор `id` должен быть положительным и уникальным, а дата создания и идентификатор генерируются сервисом. Поля `annualTurnover` и `fullName` допускают отсутствие значения; если годовой оборот задан, он должен быть больше нуля. Вложенный адрес содержит улицу длиной не более 158 символов. Тип организации задаётся перечислением `COMMERCIAL`, `TRUST`, `PRIVATE_LIMITED_COMPANY` и `OPEN_JOINT_STOCK_COMPANY`.

В XML-схемах OpenAPI структура объекта разделена на модель ответа и модель входных данных. Это позволяет явно указать автогенерируемые поля и не требовать их от клиента при создании или обновлении объекта. Для операций первого сервиса данные и ответы описаны с типом `application/xml`.

3.2 Сервис управления организациями

Спецификация `openapi/organization-service.yaml` описывает ресурс `/organizations` и дочерние ресурсы. Основные операции используют HTTP-методы по назначению: `GET` для чтения, `POST` для создания, `PUT` для полного обновления и `DELETE` для удаления.

Ресурс	Метод	Назначение
/organizations	GET	Получение коллекции с фильтрацией, сортировкой и пагинацией
/organizations	POST	Создание организации
/organizations/{id}	GET	Получение организации по идентификатору
/organizations/{id}	PUT	Полное обновление организации
/organizations/{id}	DELETE	Удаление организации
/organizations/{id}/employees	GET, POST	Получение и добавление связанных сотрудников
/organizations/{id}/employees/{employeeId}	DELETE	Удаление связанного сотрудника

Для выборки коллекции фильтры передаются как query-параметры URL и могут сочетаться. Вложенные поля обозначаются точечной нотацией, например `coordinates.x` и `postalAddress.street`. Параметр `sortBy` может повторяться для задания нескольких ключей сортировки; направление задаётся через `sortOrder`. Параметры `page` и `size` задают номер страницы и её размер.

Пример запроса:

```
GET /organizations?
type=COMMERCIAL&coordinates.x=12.5&sortBy=name&sortOrder=asc&page=1&size=10
```

Дополнительные операции статистики имеют отдельные URL: `/organizations/stats/grouped-by-name` группирует организации по имени и возвращает число элементов в каждой группе; `/organizations/stats/annual-turnover/less-than/{value}` считает организации с оборотом меньше порога; `/organizations/stats/annual-turnover/unique` возвращает уникальные значения оборота.

При некорректных данных предусмотрен ответ 400 Bad Request, при отсутствии ресурса — 404 Not Found. Для успешного создания указан код 201 Created, а удаление выполняется с ответом 204 No Content.

3.3 Сервис каталога организаций

Спецификация `openapi/orgdirectory-service.yaml` описывает второй сервис с базовым URL `/orgdirectory`. Он использует API первого сервиса и предоставляет две операции:

Путь	Метод	Назначение
------	-------	------------

<code>/filter/employees/{min-employees-count}/{max-employees-count}</code>	GET	Выборка организаций по диапазону количества сотрудников, границы включены
<code>/order/{param-name}/{desc}</code>	GET	Сортировка организаций по указанному полю в прямом или обратном порядке

Границы количества сотрудников задаются в пути. Параметр `desc` принимает `true` для сортировки по убыванию и `false` для сортировки по возрастанию. Допустимые имена полей сортировки перечислены в спецификации, включая вложенные поля. Ошибочные параметры приводят к 400 Bad Request; недоступность первого сервиса отражается ответом 502 Bad Gateway.

3.4 Интерактивная документация

Для отображения обеих спецификаций создана конфигурация Swagger UI. Скрипт `swagger/build.sh` подготавливает статический сайт и копирует в него файлы спецификаций. В интерфейсе доступны отдельные представления `Organization Service` и `OrgDirectory Service`.

Скрипт `deploy.sh` собирает документацию, передаёт статические файлы на `helios` в каталог `/home/studs/s389491/public_html/soa` и задаёт публичный адрес `https://helios.cs.ifmo.ru/~s389491/soa/`. Наличие скрипта описывает предусмотренный процесс публикации; фактическое выполнение развертывания в рамках подготовки этого отчёта не подтверждалось.

4 Результат

Подготовлены две спецификации OpenAPI версии 3.0.3, описывающие модель организации, XML-сообщения, ограничения параметров и HTTP-операции обоих сервисов. Для сервиса каталога указаны операции фильтрации по числу сотрудников и сортировки. Настроена сборка Swagger UI, которая объединяет обе спецификации в интерактивной документации.

5 Выводы

В ходе работы спроектированы REST-интерфейсы управления организациями и каталога организаций. Использование ресурсов и HTTP-методов разделяет операции по их назначению, а OpenAPI фиксирует структуру запросов и

ответов, параметры URL, ограничения и коды ошибок. Swagger UI предоставляет единый интерфейс для просмотра документации обоих сервисов.

Приложение А

Спецификация сервиса управления организациями

Спецификация OpenAPI сервиса управления организациями размещена в репозитории проекта: [organization-service.yaml](#).

Приложение Б

Спецификация сервиса каталога организаций

Спецификация OpenAPI сервиса каталога организаций размещена в репозитории проекта: [orgdirectory-service.yaml](#).